Re-Tokenization, Segmentation, and a Blind Spot: What Content-Free Boundary Markers Do in Character-Level Transformations

Anonymous authors
Paper under double-blind review

Abstract

Inserting a content-free marker (|) into the input every k characters of a character-level transformation, and asking the model to copy it through, is a cheap single-call intervention. We study what it does across seven tasks, seven open instruction-tuned models, and two versions of Gemini Flash (3.5 and 3.6), using two controls that fix tokenization at one token per character (verified at the seven open models and, through API token counts, at Gemini 3.6): the same letter tasks with spaced letters, and exact relabelings with digits. We find three separate effects. First, on letter strings, markers act largely as re-tokenization: at five of the seven open models, spacing the letters with no markers improves accuracy more (+0.16 to +0.33) than markers do (+0.13 to +0.26). Second, the gain survives one token per character: with spaced letters or digits, markers remove 33% to 83% of the remaining errors at the four strongest open models, persist to $N = 256$ at Qwen2.5-32B, and are the same there with commas or line breaks as markers. Third, with a marker every two or more characters, the position right after a marker fails whenever its output depends on the input before the marker: on four dependency tasks it falls to near chance at the stronger models while the next position recovers, and rewriting the tasks so that position needs nothing before the marker repairs it. Blocking marker-attending heads removes only part of the gain, and a two-task ablation with constrained decoding at Qwen2.5-32B shows that marking either input or output alone hurts while marking both helps; the cross-boundary failure arises from input markers alone. In practice: separate characters before adding markers, use sparse markers on long sequences on both input and output, and keep markers away from positions whose output depends on what came just before them.

1 Introduction

Ask a language model to apply a simple, fully specified rule to every character of a string (shift each letter forward, substitute through a lookup table, output the XOR of each digit with the one before) and accuracy does not fade gracefully with length. It holds near ceiling on short inputs and then collapses well before any obvious capacity limit (Hou et al., 2026). Standard remedies change the computation itself: splitting the input across several calls (Zhou et al., 2023), asking for intermediate steps (Nye et al., 2021; Chen et al., 2023), or adding memory to the architecture (Dai et al., 2019; Bulatov et al., 2022; Mohtashami & Jaggi, 2023). Each costs extra calls, extra tokens, or training.

We study a cheaper intervention. A content-free marker character (|) is inserted into the input every k characters, and the model is told to copy it through to the output unchanged. The marker carries no task information, needs no extra call, and requires no training. On letter tasks it raises accuracy substantially, by up to +0.26 on average at the stronger models we test. Our question is why, and when it hurts.

A marker does more than mark a boundary. Tokenizers merge runs of letters into multi-letter tokens, and a marker after every few letters splits those runs, so on letter strings a marker is also a tokenization intervention. We separate the two with two controls that keep one token per character whether or not markers are present:

the same letter tasks with a space between letters, and exact relabelings of them with digits. We find three effects (Figure 1):

1. **On letter strings, markers largely re-tokenize** (Section 4). At all five Qwen and Gemma models, spacing the letters with no markers at all helps more than markers on unspaced letters (+0.16 to +0.33 against +0.13 to +0.26); at Mistral and Phi the two help about equally. The striking accuracy of the position right after a marker on letter tasks (positive in all 27 task–model cells, with per-model means of +0.07 to +0.34 over the no-marker run at the same positions) is largely a token-boundary effect: with spaced letters or digits that position is no better than the rest of its segment.

2. **The gain persists when each character is its own token** (Section 5). With spaced letters or digits, markers remove 33% to 83% of the remaining errors at Qwen2.5-14B and 32B, Qwen3.8-27B, and Gemma-4-31B (significant in 23 of 24 cells with analytic standard errors; Appendix C gives the cell-level bootstrap), and the two controls agree closely. The gain is absent at short lengths, where these models are near ceiling, appears at $N = 32$, and at $N = 64$ reaches +0.27 (spaced letters) and +0.22 (digits), with little dependence on spacing. At Qwen2.5-32B it persists to $N = 256$ (56% of remaining errors removed) and is the same with commas or line breaks as markers. At the three smallest models the gains are smaller and inconsistent (significant in 7 of 18 cells).

3. **The position after a marker fails on dependencies that cross it** (Section 6). With a marker every two or more characters, on tasks where each output also depends on the previous input, the position right after a marker falls below the no-marker run in 33 of 35 task–model cells (every cell at the six strongest models), to near chance at the stronger models, even where the model is otherwise near ceiling. The next position recovers immediately, and rewriting the tasks so that this position needs nothing before the marker repairs it. The loss is not created by how the instruction describes the markers: telling the model they are only visual separators and that the rule continues across them narrows it but does not remove it, and commas or line breaks in place of | leave the position at chance up to $N = 256$. Pooled task-level effects on these tasks are consistent with a mix of this loss and the within-segment gains of Finding 2, which is why their sign varies with length and model.

We also test the obvious mechanism (Section 7). In Qwen2.5, some heads attend from each position to the adjacent marker, but at Qwen2.5-32B and $N = 64$, blocking the ten most marker-attending heads removes 1% of the gain on digits, 13% on spaced letters, and 19% on contiguous letters, against at most 5% for controls. Blocking 200 heads removes 21% (digits) and 28% (spaced letters), against none for controls of the same size. Marker-directed attention therefore contributes to the gain, and it is not concentrated in the few heads that attend to markers most; but the share removed does not rise monotonically with the number of heads blocked (8%, 1%, 28% on spaced letters), so these experiments do not establish how the contribution is distributed. A 2×2 separating input from output markers under constrained decoding, on two tasks at Qwen2.5-32B (Section 7.2), finds that marking either side alone *hurts* (−0.03 to −0.16) while marking both helps (+0.10 and +0.39), an interaction of +0.19 and +0.70; the blind spot, by contrast, follows the input markers alone. Section 8 turns the findings into guidance: separate characters before adding markers, use sparse markers on long sequences, and avoid placing a marker where an output needs the input just before it.

Supporting analyses. Each finding is backed by checks reported in the appendix: re-scoring with an edit-distance alignment, which forgives dropped or inserted characters, leaves the blind spot intact (Table 14); sampling instead of greedy decoding changes neither the gain nor the blind spot (Table 28); the per-position comparisons hold model by model and by distance from the marker (Tables 12 and 13); on third-party tasks markers help a position-local substitution task and hurt a prefix-dependent algorithm (Table 18); changing the marker symbol, jittering its placement, or rephrasing the instruction changes the size of the letter-task gain at Qwen2.5-32B but not its sign, except for a reserved special token and a terse cipher-style prompt (Appendix I); and from $N = 64$ on, sparse markers take less wall-clock time than none, because they keep models from generating past the required length (Table 33).

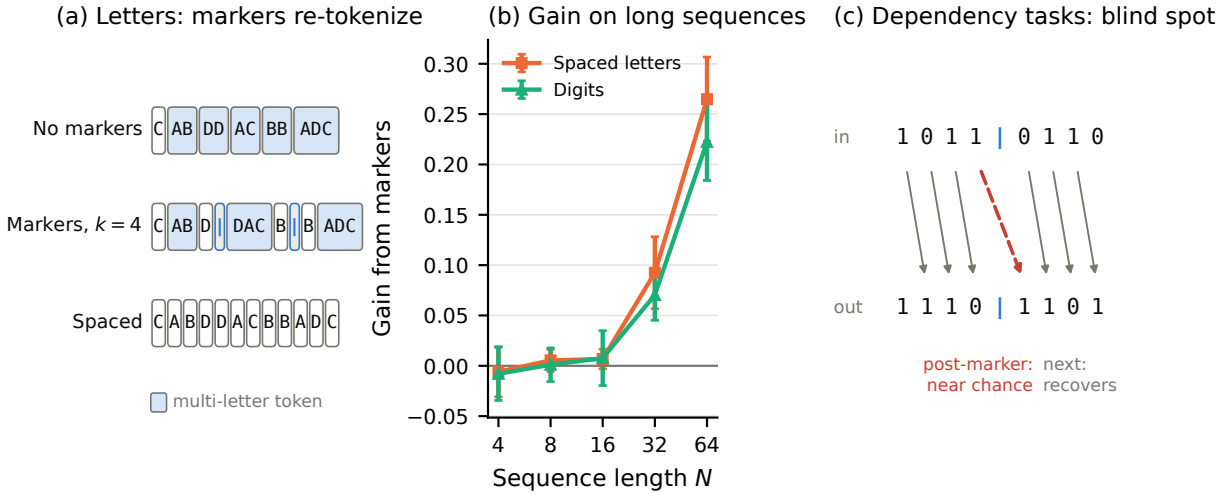


Figure 1: The three effects at a glance. (a) Qwen2.5's tokenizer merges runs of letters into multi-letter tokens; a marker every four letters splits only some of them, while spacing the letters gives one token per letter. (b) With one token per character (spaced letters or digits), markers still help the four strongest open models, from $N = 32$ on (the marker runs, "Bounds", minus the no-marker runs, "Base"; mean over models and the three position-local tasks; whiskers are 95% intervals; same data as Figure 4a). (c) On `running_xor`, each output is the XOR of its input and the one before; the position right after a marker, whose dependency crosses the marker, falls to near chance at the stronger models, and the next position recovers.

2 Related Work

Tokenization, formatting, and position. Subword tokenizers obscure the characters inside a token: models know only partially which characters their tokens contain (Kaushal & Mahowald, 2022), character-level manipulation is a known weakness (Edman et al., 2024), and on a large benchmark of character-level tasks, longer tokens obscure where a character sits (Uzan & Pinter, 2026). The same string can be encoded many ways, and models handle non-canonical encodings surprisingly well; splitting text into characters can even improve string manipulation (Zheng et al., 2025), and retokenizing a prompt while preserving its bytes is a clean probe of segmentation sensitivity (Jain et al., 2026). Instruction-tuned models cope with character-split input partly by reconstructing the canonical tokens internally from attention among the characters of each word (Yang et al., 2026), which is one reason our spaced-letter control works at all. Surface formatting matters more broadly: choices of separators, spacing, and casing in a prompt can change few-shot accuracy by tens of points (Sclar et al., 2024). How many tokens a text costs also varies sharply across languages and formats, with consequences for accuracy and price (Petrov et al., 2023; Ahia et al., 2023). Formatting and positional cues change arithmetic accuracy: Nogueira et al. (2021) show that the surface form of numbers matters, Shen et al. (2023) that positional description does, and Duan et al. (2023) that length generalization depends on how positions are presented. These works change how text is encoded and ask what the model then does. We hold the encoding fixed instead: our spaced and digit controls give one token per character whether or not markers are present, so a marker's remaining effect cannot be a change in how characters are grouped into tokens. That separation is what lets us report a re-tokenization effect and a second effect that survives it, and it is why we can attribute the post-marker failure to the boundary rather than to the tokenizer.

Learned segmentation. Tokenizer-free models learn where to place boundaries. H-Net learns content-dependent chunks over bytes end to end (Hwang et al., 2026); H-Net++ adds a small cross-chunk attention module because chunking alone ignored dependencies across chunks (Zakershahrak & Ghodratnama, 2025); and MrT5 learns to delete byte tokens partway through the encoder, merging their information into the

tokens that remain (Kallini et al., 2025). Our markers are the opposite case: fixed, content-free boundaries inserted at inference time into a model never trained to use them. The blind spot raises a question for learned chunkers too, namely whether the first position after a learned boundary also loses access to what came before it; the H-Net++ design choice suggests that dependencies across boundaries need explicit support there as well.

Attention sinks and separator tokens. Language models concentrate attention on a few tokens, most famously the first token, which act as sinks (Xiao et al., 2024; Gu et al., 2025; Zhang et al., 2025); Su et al. (2026) survey uses and interpretations. SepLLM (Chen et al., 2025) shows that separator tokens can summarize the segment before them, SinkLoRA (Zhang, 2024) exploits sinks during fine-tuning, and punctuation tokens have been used as boundary anchors for trained sparse attention (Qiu et al., 2026). Oh et al. (2024) find that tabular structure in a prompt improves factual data interactions. Those separators are learned or exploited during training, and the question is what the model has come to do with them. Ours are inserted at inference time, and no model is trained or fine-tuned for this use; the marker character is of course familiar from pretraining, but nothing ties it to these tasks. That is why the marker can be content-free: it carries no task information, so any effect must come from the boundary rather than from task content; the bar may still carry structural associations learned in pretraining, which is one reason we also test commas and line breaks. Our head ablations further show that the benefit is not carried by the heads that attend to markers most, which an attention-sink account would predict. Dangal & Shakya (2026) report a related asymmetry in natural text: inserting a comma at a clause boundary reduces degradation of later predictions, but biasing attention toward existing punctuation does not, because the inserted token brings a new hidden state computed from the full context rather than a redistribution of attention over fixed ones. Our marker is such an inserted token, and the weak effect of removing attention to it fits that reading.

Attention heads. Induction heads implement in-context copying (Olsson et al., 2022), and causal head gating assigns roles to heads by intervention (Nam et al., 2025). We characterize the marker-attending heads and block them from the markers. On character-counting probes, character-level information is encoded in early layers and then actively suppressed by late-layer components, with no attention heads specializing in routing it (Datta et al., 2026); our finding that the marker benefit is not concentrated in the heads that attend to markers most is consistent with that picture.

Long-sequence failure. Models use the middle of long contexts poorly (Liu et al., 2024), make sparse but systematic errors on flip-flop state tracking (Liu et al., 2023), and fail long-horizon tasks for reasons that compound (Wang et al., 2026; Pushkin & Abbe, 2026; Anand et al., 2026). Hou et al. (2026) quantify the accuracy cliff on repetitive deterministic tasks of the kind we use. Architectural remedies target long-sequence degradation directly; SWAN, for example, interleaves global attention layers without positional encoding with sliding-window rotary layers and extrapolates well beyond its training length (Puvvada et al., 2025). Relative position encodings such as ALiBi (Press et al., 2022) and FIRE (Li et al., 2024) likewise improve generalization to longer inputs, as do inference-time remedies that reshape attention or rescale rotary frequencies without retraining (Han et al., 2024; Wang et al., 2024); markers are a cheaper intervention still, at the prompt. Boundaries are delicate for probabilistic conditioning as well: byte-level sampling methods exist precisely because a prompt that ends mid-token distorts the next-token distribution (Hayase et al., 2026); the failure we document is a deterministic, per-position counterpart at an inserted boundary. The tokenization-independent benefit we find is a long-sequence effect of this kind.

Decomposition and memory. Least-to-most prompting (Zhou et al., 2023), program-of-thoughts (Chen et al., 2023), and scratchpads (Nye et al., 2021) decompose computation; Transformer-XL (Dai et al., 2019), recurrent memory (Bulatov et al., 2022), landmark tokens (Mohtashami & Jaggi, 2023), and learned context management (Kontonis et al., 2026) build segmentation or memory into the model. Content-free markers sit at the cheap end of this spectrum.

Benchmarks. Our tasks are generated fresh for every trial, following the contamination-resistant approach of BeyondBench (Srivastava et al., 2026). External checks use ProcBench (Fujisawa et al., 2024), the copying task of Jelassi et al. (2024), and CLRS-Text (Markeeva et al., 2026).

3 Experimental Setup

This section describes the tasks and the three formats they are posed in, the marker intervention, the models and their decoding settings, and the metrics and statistics used throughout.

3.1 Tasks and Formats

Every task maps an input string of length N to an output string of the same length under a deterministic rule stated fully in the prompt (Table 1). In *position-local* tasks, output y_i depends only on input x_i. In *one-step dependency* tasks, y_i also depends on x_{i-1}; in the *windowed* task, on x_{i-2}, x_{i-1}, x_i. For dependency tasks, the first position is copied as-is. Inputs are sampled uniformly with a deterministic seed per (task, length, spacing, alphabet size, trial), so every model sees the same inputs. `copy_previous` requires no computation, only a one-step lookback, so any harm from markers cannot be blamed on arithmetic; `sliding_window_max` uses a three-symbol window, so its output stays informative at every length.

The position-local tasks are posed in three formats. In the *letter* format (the default), letters are written contiguously, and tokenizers merge them: at each of the seven open models, whose tokenizers we can inspect, 48 random letters encode to 23 to 32 tokens without markers and to one token per letter with a marker after each letter. In the *spaced* format, the same letters are separated by single spaces and the model is asked to answer in the same format; the 48 letters encode to exactly 48 tokens at every open model whether or not markers are present, and each marker adds one token of its own. In the *digit* format, the letters are relabeled as digits ($A \to 0$, $B \to 1$, ...), which every open-model tokenizer encodes one per character. The two controls are also run on Gemini 3.6 Flash, whose tokenizer we cannot inspect but whose API reports the token count of every prompt: across its grid, each added character adds exactly one token in the spaced and digit formats at every length and spacing, and each marker exactly one more, while a letter without markers costs 0.46 tokens (Appendix Table 20). Both controls are exact transformations of the letter tasks: same seeds, same positions, same rule and prompt wording (verified in every cell). The dependency tasks use digits throughout. One caveat on `conditional_shift`: its rule depends on a symbol's position in the alphabet, which for digits equals the digit itself, so its digit version is easier.

Table 1: Tasks. A = alphabet size, M = modulus. Lengths $N \in \{4, 8, 16, 32, 64\}$. Position-local tasks are run with letters, spaced letters, and digits; dependency tasks use digits.

Task	Tier	Rule	Axis
`cyclic_shift`	position-local	next symbol in the alphabet, wrapping	$A \in \{2, 4, 8\}$
`random_permutation`	position-local	fixed arbitrary substitution (a derangement)	$A \in \{2, 4, 8\}$
`conditional_shift`	position-local	symbols at even alphabet positions forward, odd back	$A \in \{2, 4, 8\}$
`copy_previous`	one-step	$y_i = x_{i-1}$	binary
`running_xor`	one-step	$y_i = x_i \oplus x_{i-1}$	binary
`running_sum_local`	one-step	$y_i = (x_i + x_{i-1}) \bmod M$	$M \in \{2, 4, 8\}$
`sliding_window_max`	windowed	$y_i = \max(x_{i-2}, x_{i-1}, x_i)$	$A \in \{4, 8\}$

3.2 The Marker Intervention

In the marker condition ("Bounds"), a | is inserted after every k input characters, with $k \in \{1, 2, 4, 8, 16\}$; only $k < N$ is used. The expected output is the ground truth computed on the continuous, marker-free sequence, with markers spliced in at the same positions. The prompt states the rule, then instructs, for example: *"The input contains vertical bars (|). These are structural boundaries. DO NOT transform them. Copy them exactly to the output."* (the verb matches the task, e.g. "shift" for `cyclic_shift`). The no-marker baseline ("Base") uses the identical prompt, including that instruction, so that the only difference between conditions is whether markers appear in the input. That instruction is odd for an input with no bars in it, so we measured what it costs: rerunning the no-marker condition with every mention of markers removed, on the same inputs, across 300 (task, length, alphabet) cells at the four stronger models, raises accuracy by +0.006 on average (95% interval $[0.002, 0.009]$), by at most +0.026 in any model–family combination, and

the sentence helps rather than hurts in 105 of the 300 cells (Appendix C). The marker effects we report are therefore overstated by well under a percentage point on average against a baseline whose prompt never mentions bars. Example prompts are in Appendix A. Runs also included $k = 3$, which we exclude to keep the spacing grid at powers of two.

3.3 Models and Decoding

Open models are Qwen2.5-7B/14B/32B-Instruct (Qwen Team, 2024), Mistral-7B-Instruct-v0.3 (Jiang et al., 2023), Phi-3.5-mini-instruct (Abdin et al., 2024), Qwen3.8-27B (Qwen Team, 2026), and Gemma-4-31B-it (Gemma Team, 2026), run locally in bfloat16 with each model's chat template and no system message beyond the template's own default (thinking disabled for Qwen3.8 and Gemma 4) and greedy decoding. Runs use PyTorch 2.6.0 (CUDA 12.4) and Transformers 5.12.1 with the eager attention implementation, on NVIDIA A100 80GB GPUs (Titan RTX for some smaller-model runs); Appendix B lists the exact checkpoint revisions. Qwen3.8-27B is a hybrid architecture: 48 of its 64 layers use Gated DeltaNet linear attention (Yang et al., 2025) and only 16 use softmax attention; Gemma-4-31B-it uses softmax attention in every layer (50 sliding-window and 10 global layers). The generation budget is $1.5N + 20$ new tokens, or $2N + 20$ at $k = 1$, where a correct marked answer is $2N - 1$ tokens. Predictions are clipped to the answer's length before scoring, so a budget can matter only by cutting an answer short; among the 86,400 generations whose raw output we stored, this happened in at most 0.6% of any condition class, at similar rates with and without markers, and in at most 0.04% at the four stronger models (Appendix B).[1] The closed models are two versions of Gemini Flash, queried through the API with the same prompts and seeds; the resolved model name and timestamp are stored with every generation. Gemini 3.5 Flash (Google, 2026a) was run on the letter and dependency tasks, but not on the two controls, between 2026-08-11T23:28 and 23:45 UTC at temperature 0 with thinking disabled (`thinking_budget` 0), the closest available match to the open models' greedy decoding, under the versioned model name rather than the `-latest` alias, and every response reported `gemini-3.5-flash` as the model served; its usage metadata likewise reports a total equal to prompt plus output tokens on all 2,040 calls. On 2026-10-08 Google deprecated `gemini-3.5-flash` and began routing its requests to `gemini-3.6-flash` (Google, 2026c), so the controls could not be added at that model; instead we ran the full grid of all 13 task–format variants (the seven tasks with both controls) on Gemini 3.6 Flash (Google, 2026b) between 2026-11-12T07:56 and 10:00 UTC with 5 trials per cell. Every request sent `thinking_budget` 0 (the runner probes the setting once per model name and reuses it for every call), which this model accepts: on every one of the 4,930 calls the usage metadata reports no thought tokens and a total equal to prompt plus output tokens, and every answer is full length, whereas a request at the model's default thinking level spends about 160 thought tokens and, under our output budget, truncates the answer. Gemini 3.6 Flash deprecates and ignores the temperature parameter, so its outputs are samples at the model's fixed settings rather than greedy decodes: repeating an identical request four times with thinking off gave two to four distinct outputs on each of three cells we checked. Its trials are therefore independent samples as well as independent inputs. Qwen2.5-32B is our primary model for head-level experiments.

3.4 Metrics and Statistics

Character accuracy is the fraction of output positions whose character matches the ground truth, after stripping markers and spaces from both sides and clipping the prediction to the ground-truth length. We avoid the name "token accuracy" deliberately: the unit is a character of the answer, not a tokenizer token. Marker placement is not part of this metric; Appendix L reports exact transformation match and marker-placement compliance separately.

Pooled effects. Only spacings $k < N$ are run, so marker runs are concentrated at longer lengths than no-marker runs, and accuracy falls with length. A pooled effect therefore compares the mean character accuracy of the marker runs with the no-marker mean reweighted to the same length distribution, $\Delta = \overline{\mathrm{acc}}_{\mathrm{Bounds}} - \sum_N p_N \, \overline{\mathrm{acc}}_{\mathrm{Base},N}$, where p_N is the share of marker runs at length N; it pools $N \le 64$, $k \in \{1, 2, 4, 8, 16\}$,

[1] The $k = 1$ cells at $N \ge 64$ were first generated under the smaller budget, which cut them off, and then regenerated; with greedy decoding the new outputs extend the old ones, except for Mistral, Phi, and Qwen2.5-7B on two tasks, whose original runs used a different GPU model and diverge after a shared prefix with the same accuracy over it.

and alphabet or modulus values, and z is Δ over its standard error. Because the controls leave strong models near ceiling, we also report the *share of remaining errors removed*: the mean Δ over a model's tasks divided by one minus the mean length-matched no-marker accuracy. Open models use 20 to 80 trials per cell (Appendix B); Gemini 3.5 uses 3 and Gemini 3.6 uses 5. We call a pooled cell significant at $|z| \geq 3.54$, the Bonferroni-corrected two-sided $p < 0.05$ across all 125 pooled cells we test (the 77 in the overview of Section 6 and 48 for the two controls).

Units of uncertainty. Every generation uses its own input, drawn from a seed determined by (task, length, spacing, alphabet, trial). Because the seed includes the spacing, a marker run and the no-marker run it is compared with see *different* random inputs from the same distribution: comparisons are between conditions, not paired by input. Two follow-up experiments are paired by construction, the input/output-marker 2×2 (Section 7.2) and the clean-baseline check (Appendix C); the main grids are not, and a paired rerun of them would reduce variance but was not done. The unit of uncertainty is therefore the generation, and standard errors are the usual $s/\sqrt{n}$ over generations, combined across the two conditions as $\mathrm{SE}(\Delta) = \sqrt{\mathrm{SE}^2_{\mathrm{Bounds}} + \sum_N p_N^2 \, \mathrm{SE}^2_{\mathrm{Base},N}}$, with $z = \Delta/\mathrm{SE}(\Delta)$. A pooled cell mixes lengths, spacings, and alphabet values of differing difficulty, so Appendix C repeats every pooled cell with two bootstraps. Resampling generations supports the pooled effects (the Bonferroni verdict changes in 1 of 91 cells); the more conservative bootstrap over whole experimental configurations is much wider, leaving 16 of 91 cells above the threshold instead of 62, largely because the grid deliberately spans conditions where the effect is absent and because each cell contains few configurations. We therefore read pooled significance as conditional on the grid we ran, and rely on the length-resolved and position-resolved analyses to establish the structure of the effects.

Redundant configurations. At $A = 2$ the three position-local rules coincide: the only derangement of two symbols, the cyclic successor, and the conditional shift all flip the bit. At $M = 2$, `running_sum_local` computes `running_xor`. These cells still test different *instructions* for the same function, but they are not independent evidence across tasks; Appendix C recomputes every pooled effect with them dropped.

Per-position accuracy. For a marker run with spacing k, output position i (0-indexed, markers stripped) is *post-marker* if $i > 0$ and $i \bmod k = 0$, and *within-segment* otherwise. Each class is compared with the no-marker run's accuracy at the same position indices, which controls for accuracy varying along the sequence. Per-position results pool $N \leq 64$, $k \in \{2, 4, 8, 16\}$, and alphabet or modulus values; $k = 1$ has no within-segment positions, so per-position claims concern spacings of two or more. The unit for uncertainty is the generation, and intervals are 95%.

4 Tokenization Effects on Letter Strings

Figure 2 compares the three formats. On contiguous letters, markers raise accuracy at every model: by +0.26 at Qwen3.8-27B, +0.24 at Qwen2.5-32B, +0.23 at Gemma-4-31B, +0.20 at Qwen2.5-14B, +0.13 at Qwen2.5-7B, +0.07 at Mistral-7B, +0.06 at Phi-3.5-mini, and +0.22 at both Gemini versions (all 27 cells positive, 25 significant). But at the five Qwen and Gemma models, and at Gemini 3.6, simply spacing the letters, with no markers, helps more: +0.33 at Qwen3.8, +0.32 at Gemma 4 and Gemini 3.6, +0.30 at Qwen2.5-32B, +0.27 at Qwen2.5-14B, and +0.16 at Qwen2.5-7B. At these models, spaced letters without markers are more accurate than contiguous letters with markers (for example, 0.80 against 0.74 at Qwen2.5-32B, 0.89 against 0.79 at Gemma 4, and 0.97 against 0.87 at Gemini 3.6); at Mistral (+0.06 from spacing) and Phi (+0.05), spacing and markers help about equally. Without markers, the letter tasks are hard for these models largely because of how letters are tokenized: averaged over lengths, the four stronger models solve the digit versions at 0.78 to 0.93, against 0.44 to 0.56 with contiguous letters.

The per-position view (Figure 3) shows how. On contiguous letters, the position right after a marker beats the no-marker run at the same positions in all 27 task–model cells, with per-model means from +0.07 (Phi) to +0.34 (Qwen2.5-32B, Qwen3.8, and Gemini 3.6), and it beats the within-segment positions of the same marker runs in 25 of 27 cells. A marker gives the following letter its own token. With spaced letters or digits, where every symbol is already its own token, that advantage is gone: the post-marker position beats the within-segment positions in only 12 of 24 cells in each control, and the two classes are within 0.04 of each

other at every model, with the post-marker position slightly the lower of the two at Qwen2.5-7B (Appendix Table 12). That is pooled over spacings; at the sparsest spacing, $k = 16$, the post-marker position on the controls falls 0.07 to 0.16 below the position after it at Qwen2.5-7B, Mistral, and Phi, and on spaced letters at Qwen2.5-14B (Appendix Table 13), a small echo of the dependency-task loss that the stronger models do not show. The spacing profile tells the same story (Figure 4b, next section): on contiguous letters at $N = 64$, the gain is largest with a marker after every letter (+0.47 at the four stronger models) and falls steadily as markers get sparser (+0.09 at $k = 16$), as expected if markers help by splitting letter runs into shorter tokens. And the letter gain is present even on four-letter strings (+0.15), where the controls show no marker effect at all.

5 Tokenization-Independent Gains on Long Sequences

Holding each character to one token does not hold the whole prompt fixed: markers still add tokens, lengthen the sequence, and shift every later position. What the controls establish is narrower, that the gain is not an artifact of merging characters into multi-character tokens; we call it *tokenization-independent* in that sense throughout. With tokenization fixed, markers still help the stronger models. With spaced letters, the gain is +0.13 at Qwen2.5-32B, +0.11 at Qwen2.5-14B, +0.09 at Qwen3.8, and +0.06 at Gemma 4, removing 62%, 38%, 55%, and 48% of their remaining errors; with digits it is +0.06 to +0.09, removing 33% (Qwen2.5-14B) to 83% (Gemma 4). These effects are significant in all 12 spaced-letter cells and 11 of 12 digit cells at these four models. At the three smallest models the gains are smaller and inconsistent: with spaced letters, +0.02 (Qwen2.5-7B), +0.06 (Mistral), and +0.03 (Phi); with digits, +0.05, −0.01, and +0.10; significant in 7 of these 18 cells. Gemini 3.6 Flash is at 0.94 to 1.00 on both controls without markers, so they leave it no room to show the effect: markers add 0.00 to +0.04 there, none significant; in the one control cell where it is below 0.95 without markers, spaced letters at $N = 64$ (0.90), they add +0.09.

This benefit is a long-sequence effect (Figure 4a). Up to $N = 16$, the four stronger models are near ceiling in both controls (0.92 to 0.96 without markers) and markers change nothing. At $N = 32$, accuracy without markers starts to fall and markers add +0.07 to +0.09; at $N = 64$, where accuracy without markers is 0.59 (spaced letters) and 0.68 (digits), markers add +0.27 and +0.22. In the strongest case, Qwen2.5-32B on `random_permutation` with eight digits at $N = 64$, accuracy rises from 0.41 to between 0.87 and 0.99 with

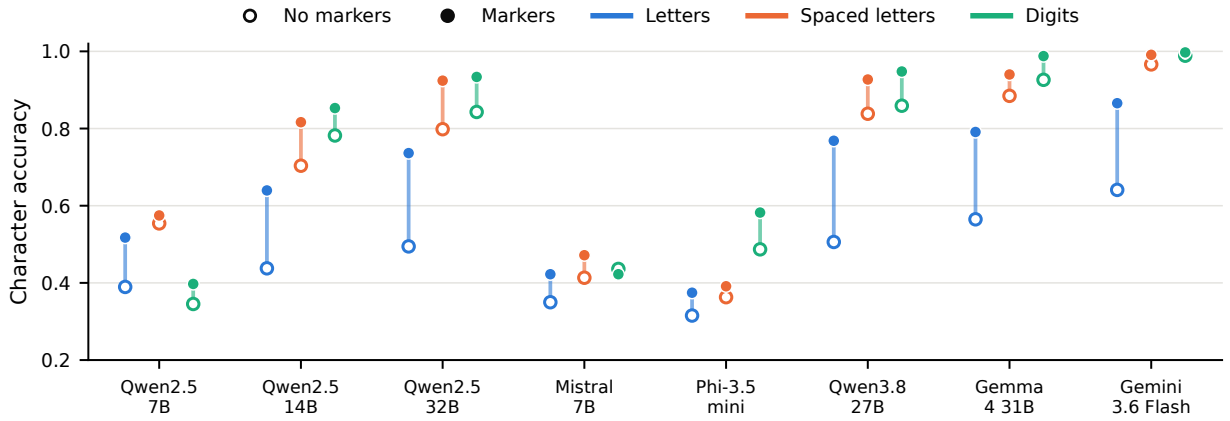


Figure 2: Character accuracy without markers (hollow) and with markers (filled) on the three position-local tasks in three formats: contiguous letters (blue), spaced letters (orange), and digits (green). Mean over tasks; $N \leq 64$, $k \in \{1, 2, 4, 8, 16\}$; no-marker values are length-matched to the marker runs (Section 3); whiskers (95% intervals) are mostly hidden by the markers. At the Qwen and Gemma models, spacing the letters without markers beats markers on contiguous letters. With tokenization fixed, markers still help the stronger models.

any spacing from every 2 to every 16 characters. The gain depends little on spacing (Figure 4b): at $N = 64$ it is +0.21 to +0.32 for spaced letters and +0.17 to +0.26 for digits across all spacings, slightly weaker at the extremes (a marker after every character, which doubles the length of the output, or after every 16).

The gain is spread over the whole segment rather than concentrated after the marker (Figure 3, middle). A similar tokenization-independent gain appears inside segments on the dependency tasks, which use digits: within-segment positions gain +0.07 at Qwen2.5-32B, +0.05 at Gemma 4 and Gemini 3.5, and +0.02 at Qwen3.8 and Gemini 3.6 over the no-marker run at the same positions, though not at Qwen2.5-14B. Under edit-distance alignment these within-segment gains shrink but stay positive at the stronger models (for example, Qwen2.5-32B: +0.11 to +0.03 on `running_xor` and +0.11 to +0.05 on `running_sum_local`; Qwen3.8: +0.13 to +0.06 and +0.14 to +0.09), so part of what markers do inside segments is keep the model from slipping out of alignment. The gap between the two scorings is the alignment-sensitive part of the gain, the part that index-by-index scoring attributes to fewer alignment slips; it is 52% of the within-segment gain in the median dependency-task cell at the four stronger models (range 14% to 85%;

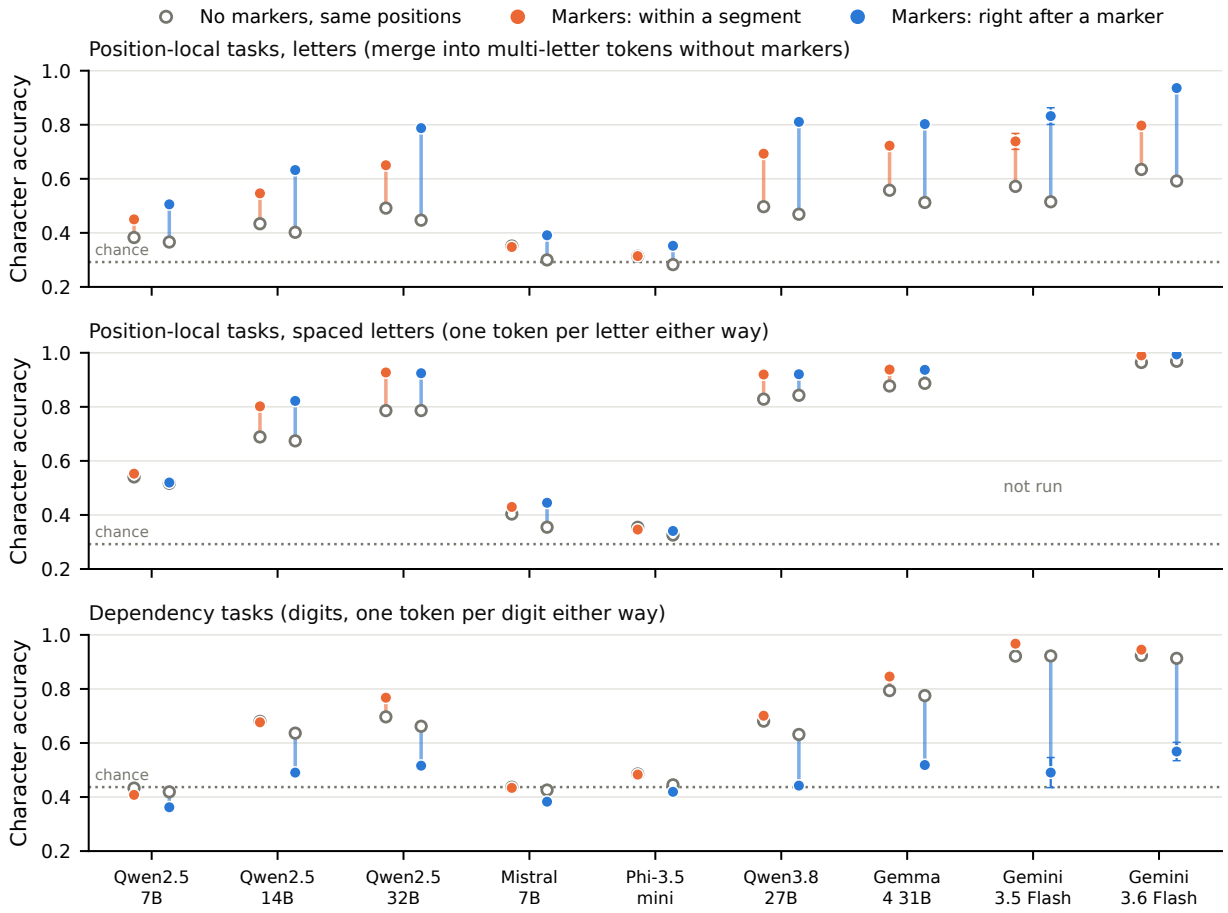


Figure 3: Per-position accuracy, every model. For each model: accuracy within a segment (orange) and at the position right after a marker (blue), each next to the no-marker run's accuracy at the same position indices (hollow gray). Mean over the tasks in each block; $N \leq 64$, $k \in \{2, 4, 8, 16\}$; whiskers are 95% intervals; dotted line: chance (always guessing the most frequent output symbol), averaged over the tasks. Top: on contiguous letters, the post-marker position gains the most. Middle: with spaced letters it is no better than the rest of its segment. Bottom: on dependency tasks it loses at every model (Section 6). Gemini 3.5 was not run on spaced letters.

`results_clean/_summary/error_taxonomy.csv`). The remainder survives alignment and so reflects fewer substitutions, though the split between two scoring rules is a descriptive decomposition, not a causal one. Markers also change how errors cluster along the sequence (Appendix Table 15): at $N = 64$ under index scoring, the probability that an error is followed by another error is 0.70 to 0.77 without markers on the spaced and digit formats at the four stronger models and 0.24 to 0.59 with a marker every 2 to 16 characters, and the mean run of consecutive errors shortens from 3.1 to 3.9 positions to 1.3 to 2.3; under alignment the same probability moves only from 0.34 to 0.51 to 0.21 to 0.40, so what sparse markers mostly shorten is the cascade after a slip. A marker after every character lowers the error rate but leaves that probability at 0.45 to 0.70. The gain also extends beyond the lengths of our main grid: at Qwen2.5-32B, on `random_permutation` with digits ($A = 8$, $k \in \{4, 16\}$), markers remove 89% to 91% of the remaining errors at $N = 64$, 78% to 81% at $N = 128$, and 56% to 57% at $N = 256$ (accuracy 0.18 without markers, 0.64 to 0.65 with), the ranges spanning |, commas, and line breaks as markers (Appendix Table 27). One reading is that a marker every few characters helps a strong model keep its place in a long sequence; Section 7 states that hypothesis precisely and tests the simplest version of it. The experiments in this section establish the effect, not its cause. Section 7 shows that the heads that attend most to markers carry little of this effect, and Qwen3.8-27B, whose layers are mostly linear attention, shows it comparably to the other strong models.

6 The Post-Marker Blind Spot on Dependency Tasks

On the four dependency tasks, with a marker every two or more characters, the position right after a marker is below the no-marker run at the same positions in 33 of 35 task–model cells (Figure 3, bottom; these tasks use digits, so tokenization plays no role). The two exceptions are slight (+0.02 each, Qwen2.5-7B on `running_xor` and Phi on `sliding_window_max`); every cell at Qwen2.5-14B and 32B, Qwen3.8, Gemma 4, and both Gemini versions shows the loss. We call this the *blind spot* for short. The term describes the behavior, not a claim about what the model represents: accuracy near chance shows that the information before the marker is not used correctly here, not that it is inaccessible to the model. The loss grows with model strength: −0.03 to −0.06 at Phi, Mistral, and Qwen2.5-7B; −0.15 at Qwen2.5-14B and 32B; −0.19 at Qwen3.8; −0.26 at Gemma 4; and −0.43 and −0.34 at Gemini 3.5 and 3.6, which are near ceiling without markers. At the stronger models the post-marker position is close to chance: on `running_xor` (chance 0.5) it is correct 0.51 to 0.55 of the time at Qwen2.5-14B and 32B, Qwen3.8, Gemma 4, and both Gemini versions. The failure is confined to the positions whose inputs lie across the marker. On the one-step tasks that is one

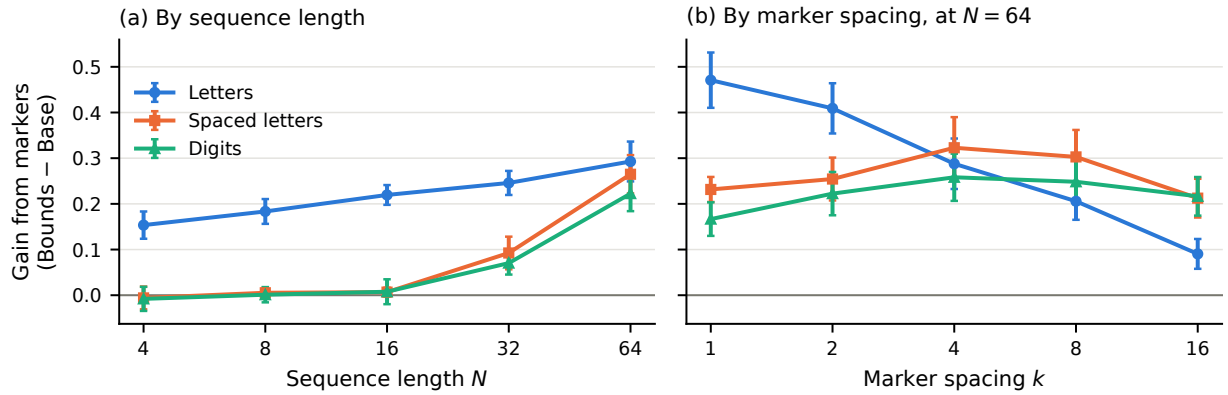


Figure 4: The marker effect (Bounds − Base) at the four stronger models (Qwen2.5-14B and 32B, Qwen3.8-27B, Gemma-4-31B), mean over the three position-local tasks. (a) By sequence length, pooled over spacing. (b) By marker spacing at $N = 64$. Whiskers are 95% intervals over the 12 model–task pairs. The two tokenization controls (spaced letters, digits) agree: no effect up to $N = 16$, a large effect at $N = 64$, and little dependence on spacing. Contiguous letters show a different signature: a gain at every length that is largest for the densest markers.

position: on `running_xor` at $k \geq 4$ the change against the no-marker run is -0.15 at the position right after a marker and $+0.20$ at the next, averaged over the four stronger models. `sliding_window_max` needs the two previous inputs, and there the first *two* positions are affected, -0.13 and $+0.02$, with recovery ($+0.13$) only at the third (`results_clean/_summary/blind_spot_by_distance.csv`). We count that second position as within-segment throughout, which makes the dependency-task estimates conservative for that task. With a marker every 16 characters, accuracy one position later is back at 0.85 (Qwen2.5-32B), 0.70 (Qwen3.8), 0.82 (Gemma 4), 1.00 (Gemini 3.5), and 0.91 (Gemini 3.6). A marker after every character ($k = 1$) is different: there every position follows a marker, and on `copy_previous` at $N = 32$ Qwen2.5-32B is more accurate with such markers (0.88) than without (0.83), plausibly because a fully regular pattern makes the lookback fixed (Section 7).

Alignment-robust scoring. Character accuracy compares characters index by index, so a dropped or inserted character misaligns everything after it. Re-scoring every run with an edit-distance alignment, which repairs such slips instead of counting the rest of the output as wrong, leaves the post-marker loss in place at every strong open model and task (Appendix Table 14). At Qwen2.5-32B it is -0.24 on `running_xor` and -0.22 on `running_sum_local` under alignment, against -0.19 and -0.16 index by index.

Intermediate spacings. The per-position analyses use spacings that are powers of two. The grids also include $k = 3$, and repeating the comparison at $k = 2$, 3, and 4 separately gives a mean post-marker change of -0.20 at each of the three spacings over the sixteen dependency-task cells at the four stronger models, with $k = 3$ matching the sign of its neighbours in 15 of 16 (Appendix Table 16). The loss is a property of the position after a marker, not of a rhythm in the grid.

Instruction wording. Our instruction calls the bars "structural boundaries", which might invite the model to treat each segment as a fresh start. We reran `running_xor` and `running_sum_local` at Qwen2.5-32B, Qwen3.8, and Gemma 4 with two other wordings (Table 2): a *neutral* one that only asks for each bar to be copied through, and an *across* one stating that the bars are only visual separators and that the digit right after a bar uses the digit right before it as its previous input. The neutral wording changes the post-marker loss by at most 0.02. The across wording narrows it, most at Gemma 4 (from -0.41 to -0.15 on `running_sum_local` and from -0.26 to -0.15 on `running_xor`) and least at Qwen2.5-32B (-0.16 to -0.14 and -0.19 to -0.14), but in every cell the post-marker position stays below the no-marker run, and at Qwen2.5-32B it remains near chance on `running_xor` (0.58; chance 0.5). The framing does not create the blind spot, and an instruction that names the dependency explicitly repairs it only in part. A third variant adds to the across wording two worked examples with bars ($N = 8$, $k = 2$ and 4, produced by the task's own rule), the first annotated at the position right after its first bar, and we ran it on all four dependency tasks (Table 3). The demonstration does what the instruction alone did not, at one model: at Gemma 4 the post-marker loss goes from -0.29 to 0.00 on `copy_previous`, from -0.41 to -0.07 on `running_sum_local`, from -0.26 to -0.13 on `running_xor`, and from -0.07 to $+0.14$ on `sliding_window_max`. At Qwen2.5-32B it repairs less (-0.15 to -0.13, -0.16 to -0.09, -0.19 to -0.12, and -0.09 to -0.03), and its post-marker position on `running_xor` stays near chance (0.57). At Qwen2.5-7B, near chance on these tasks with or without markers, nothing changes except on `sliding_window_max` (-0.11 to $+0.02$). At the two stronger models the within-segment gains move by at most 0.05 and the no-marker baselines by at most 0.04 under the new wording. So the blind spot is not a matter of framing, and naming the dependency repairs it only in part, but a worked demonstration of the crossing step removes most of it at Gemma 4 and about a third at Qwen2.5-32B: how far a model can be talked out of it depends on the model.

Other delimiters and longer sequences. At Qwen2.5-32B we replaced `|` with a comma or a line break, each with a constraint sentence naming it, and extended `running_xor` and `running_sum_local` (modulus 4) to $N = 128$ and 256 ($k \in \{4, 16\}$, 20 trials per cell; Appendix Table 27). With every delimiter and at every length, the post-marker position is at chance: 0.48 to 0.52 on `running_xor` (chance 0.5) and 0.21 to 0.26 on `running_sum_local` (chance 0.25). Within-segment positions gain $+0.16$ to $+0.26$ and $+0.09$ to $+0.19$ over the no-marker run at the same positions. Measured against the no-marker run, the post-marker loss shrinks with length (on `running_xor`, -0.11 to -0.14 at $N = 64$, -0.03 to -0.05 at 128, and $+0.03$ to $+0.05$ at 256), but only because the no-marker run itself decays toward chance on long sequences; the post-marker

Table 2: Instruction wording and the blind spot: per-position change from adding markers (marker run minus the no-marker run of the same wording, at the same positions). Original: the bars are "structural boundaries". Neutral: copy each bar through. Across: the bars are only visual separators and the rule applies across them. $N \leq 64$, $k \in \{2, 4, 8, 16\}$, 20 trials per cell.

		Post-marker			Within-segment		
Task	Model	Original	Neutral	Across	Original	Neutral	Across
`running_xor`	Qwen2.5 32B	−0.19	−0.17	−0.14	+0.11	+0.06	+0.09
`running_xor`	Qwen3.8 27B	−0.06	−0.07	−0.03	+0.13	+0.07	+0.13
`running_xor`	Gemma 4 31B	−0.26	−0.25	−0.15	+0.09	+0.07	+0.03
`running_sum_local`	Qwen2.5 32B	−0.16	−0.15	−0.14	+0.11	+0.07	+0.09
`running_sum_local`	Qwen3.8 27B	−0.06	−0.07	−0.04	+0.14	+0.13	+0.14
`running_sum_local`	Gemma 4 31B	−0.41	−0.39	−0.15	+0.11	+0.08	+0.07

Table 3: Worked examples and the blind spot: no-marker accuracy and the per-position change from adding markers (marker run minus the no-marker run of the same wording at the same positions) under the original wording and under the across wording plus two worked examples with bars, the first annotated at the position after its first bar. Four dependency tasks, $N \leq 64$, $k \in \{2, 4, 8, 16\}$, 20 trials per cell.

		No-marker acc.		Post-marker		Within-segment	
Task	Model	Original	Worked	Original	Worked	Original	Worked
`copy_previous`	Qwen2.5 7B	0.53	0.65	−0.10	−0.10	−0.07	−0.05
	Qwen2.5 32B	0.86	0.88	−0.15	−0.13	−0.02	+0.00
	Gemma 4 31B	0.93	0.91	−0.29	−0.00	−0.07	−0.07
`running_xor`	Qwen2.5 7B	0.41	0.34	+0.02	+0.03	+0.01	+0.06
	Qwen2.5 32B	0.78	0.76	−0.19	−0.12	+0.11	+0.09
	Gemma 4 31B	0.84	0.86	−0.26	−0.13	+0.09	+0.04
`running_sum_local`	Qwen2.5 7B	0.31	0.30	−0.03	−0.02	−0.01	+0.03
	Qwen2.5 32B	0.59	0.58	−0.16	−0.09	+0.11	+0.11
	Gemma 4 31B	0.80	0.79	−0.41	−0.07	+0.11	+0.09
`sliding_window_max`	Qwen2.5 7B	0.60	0.56	−0.11	+0.02	−0.02	+0.01
	Qwen2.5 32B	0.68	0.72	−0.09	−0.03	+0.08	+0.05
	Gemma 4 31B	0.74	0.74	−0.07	+0.14	+0.08	+0.08

position does not improve. The blind spot is a property of the boundary, not of the symbol. The line break gives the largest pooled gain in all six dependency cells, by up to 0.08.

Input markers alone. Markers in the input with an output asked for without them leave the post-marker position at chance on `running_xor` (Section 7.2), so the failure follows the segmentation of the input, not the request to emit markers.

Prompt delivery. Qwen2.5's chat template inserts a system prompt of its own when none is given, so every run above carries one. Delivering the same prompts three other ways at Qwen2.5-32B leaves both effects in place (Appendix Table 30): with an empty system message, a task-oriented system message, or no chat template at all, the gain on `random_permutation` with digits is +0.36 to +0.39 against +0.37 by default, and on `running_xor` the post-marker position stays at chance (0.51 to 0.53, chance 0.5) while within-segment positions gain +0.14 to +0.18. These results suggest that the effects are robust to the prompt-delivery formats tested, including removing the chat template. The checkpoint remains instruction-tuned without its template, so whether the effects persist in base models remains untested.

Larger state spaces. The dependency tasks above use at most eight symbols. We reran `running_sum_local` at Qwen2.5-32B with modulus 8, 16, and 32, writing the symbols space-separated so that each is one token at every modulus (Appendix Table 29). The post-marker position sits at chance

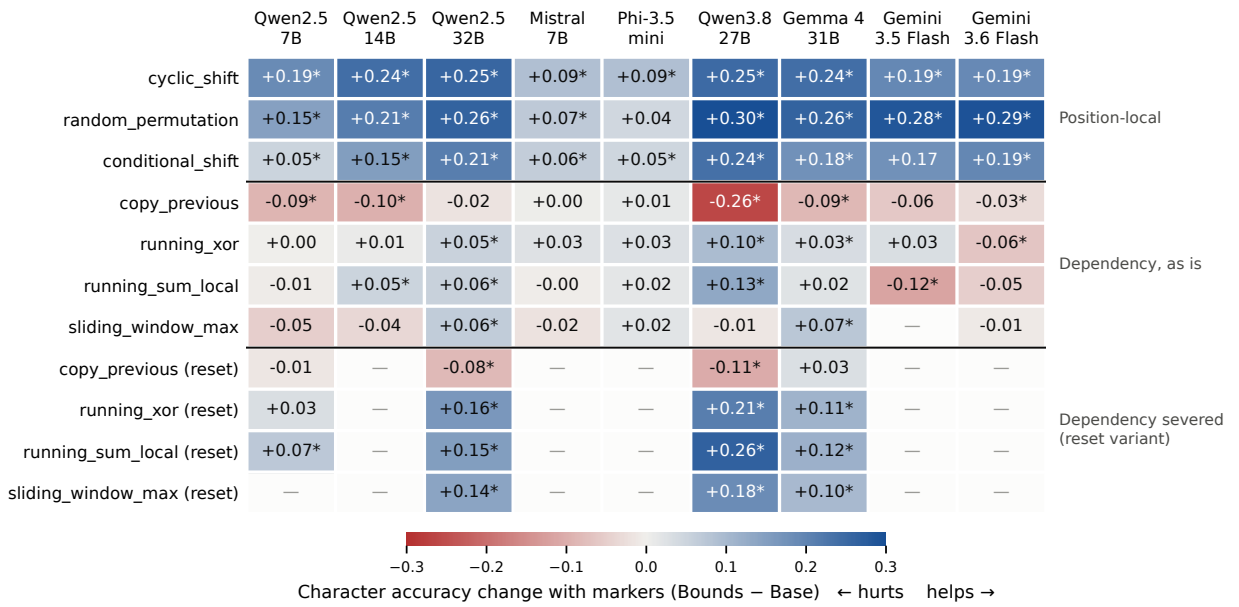


Figure 5: Pooled change in character accuracy when markers are added (Bounds − Base), contiguous-letter position-local tasks and digit dependency tasks, every model. Pooled over $N \in \{4, \ldots, 64\}$, $k \in \{1, 2, 4, 8, 16\}$, and alphabet or modulus values, with the no-marker mean length-matched to the marker runs. *: $|z| \geq 3.54$ (Bonferroni across the 125 pooled cells we test). Blue: markers help; red: markers hurt; —: not run. Bottom: dependency tasks rebuilt so the position after each marker needs nothing before it (Section 6.1).

at every modulus: 0.13 against a chance level of 0.125 at $M = 8$, 0.054 against 0.062 at $M = 16$, and 0.043 against 0.031 at $M = 32$, while within-segment positions gain +0.03 to +0.08 over the no-marker run at the same positions. The blind spot is therefore not an artifact of a small alphabet. One caveat bounds the largest case: without markers the model reaches only 0.075 at $M = 32$ against a chance level of 0.031, so that column is compressed toward the floor.

Sampled decoding. All open-model runs above decode greedily (Gemini 3.6, which is sampled, shows the blind spot too; Section 3). Repeating three tasks at Qwen2.5-32B with sampling ($N \in \{32, 64\}$, $k \in \{4, 16\}$; temperature 0.7 with top-p 0.8, and temperature 1.0 with top-p 0.95; one sample per prompt) changes little (Appendix Table 28). The post-marker position stays at chance, 0.51 to 0.52 on `running_xor` and 0.26 to 0.27 on `running_sum_local` under all three decoding regimes, and the gain on `random_permutation` with digits is +0.37 greedy, +0.38 at temperature 0.7, and +0.42 at temperature 1.0.

Pooled effects on dependency tasks. Averaged over all positions (Figure 5), a dependency task's response to markers combines this post-marker loss, on about a fraction $1/k$ of positions, with the within-segment gain of Section 5 on the rest, and the pooled sign varies accordingly: of 35 dependency cells, 8 are significantly positive (all on `running_xor`, `running_sum_local`, or `sliding_window_max` at the stronger open models) and 7 significantly negative (five on `copy_previous`). Averaged over tasks, the effect ranges from −0.05 (Gemini 3.5) and −0.04 (Gemini 3.6 and Qwen2.5-7B) to +0.04 (Qwen2.5-32B). `copy_previous`, which needs no computation, is negative at seven of nine models, significantly at five, including where the model is near ceiling without markers (Qwen3.8: −0.26). The pattern with length is consistent with this mixture: short sequences admit only dense spacings ($k < N$), so a larger share of their positions sit right after a marker, while long sequences add sparse spacings and are where the within-segment gain is large. At Qwen2.5-32B, `running_xor` goes from −0.11 at $N = 4$ to +0.13 at $N = 64$, and `running_sum_local` from −0.05 to +0.14; Gemma 4's `running_sum_local` goes from −0.10 at $N = 8$ to +0.19 at $N = 64$. Qwen3.8 gains most on balance on `running_xor` and `running_sum_local` (+0.10 and +0.13): its post-

marker positions fail as at every model, but its within-segment positions on those two tasks gain +0.13 and +0.14.

6.1 Reset Variants

If the post-marker position fails because it needs the input before the marker, removing that need should repair exactly that position and leave the others alone. We rebuild each dependency task as a *reset* variant: the position right after each marker is treated like the first position of the sequence, so the recurrence restarts at every marker. The prompt states the new rule with a worked example (Appendix A). Reset variants were run at Qwen2.5-7B and 32B, Qwen3.8-27B, and Gemma-4-31B.

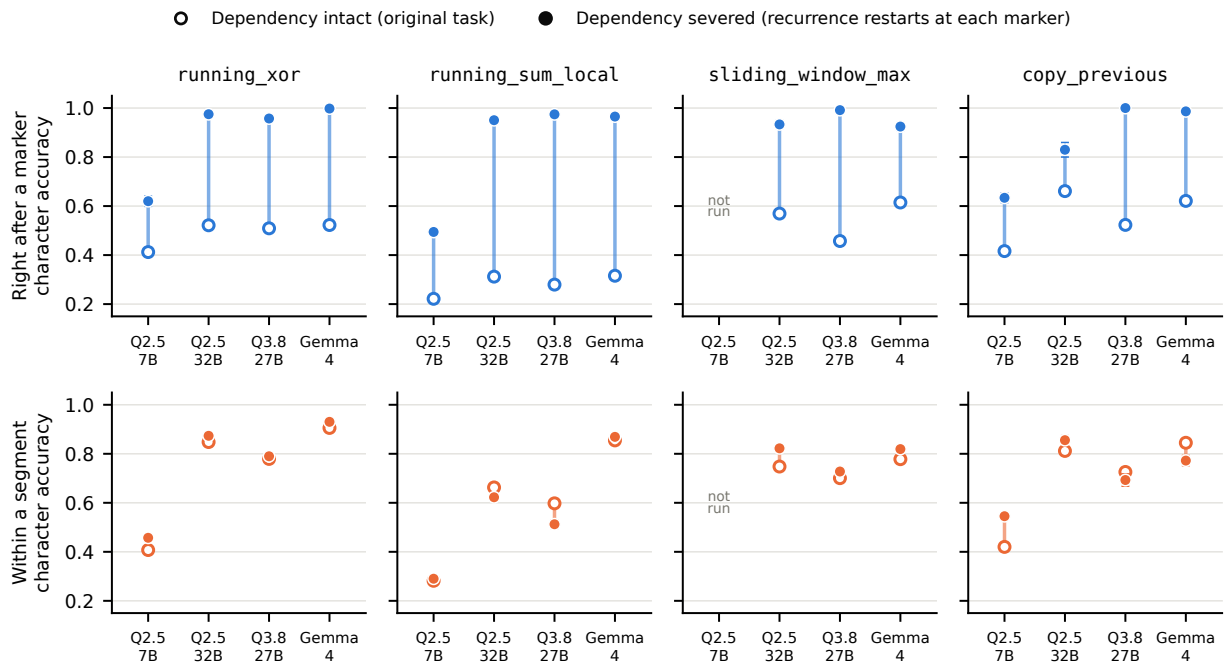


Figure 6: Reset variants, per position. Top row: accuracy at the position right after a marker, with the dependency intact (hollow) and with the recurrence restarting at each marker (filled). Bottom row: the same for within-segment positions. Models: Qwen2.5-7B and 32B, Qwen3.8-27B, Gemma-4-31B; `sliding_window_max` was not run in reset form at 7B. $N \leq 64$, $k \in \{2, 4, 8, 16\}$; whiskers are 95% intervals.

Figure 6 confirms the prediction. On `running_xor`, `running_sum_local`, and `sliding_window_max` at the three larger models, the post-marker position rises from 0.28–0.61 to 0.92–1.00, while within-segment positions change by −0.09 to +0.07. Qwen2.5-7B moves the same way with smaller gains (`running_xor` 0.41 to 0.62; `running_sum_local` 0.22 to 0.49). Two caveats bound what this shows. The reset rule makes the post-marker position easier by design (on `running_xor` it becomes "copy this digit"), so the experiment localizes the harm rather than showing a new benefit. And the reset prompt alone shifts no-marker accuracy by −0.08 to +0.06, so pooled reset effects compare different baselines (Appendix Table 17); the per-position comparison uses marker runs only.

`copy_previous` behaves differently. Its post-marker position is also repaired (0.52–0.66 to 0.83–1.00), but its within-segment positions fall at Qwen3.8 (−0.03) and Gemma 4 (−0.07), and in the original task they are below the no-marker run at eight of the nine models including both Gemini versions, with Mistral level (per-task values in `results_clean/_summary/per_position_values.csv`; the open models' values also appear in Appendix Table 14). Because the output is a shifted copy of the input, an off-by-one slip at a marker

can carry through the rest of the segment; we read this as desynchronization, a failure distinct from the post-marker loss. Edit-distance alignment supports this reading: once slips are repaired, `copy_previous`'s within-segment loss shrinks from -0.24 to -0.05 at Qwen3.8 and from -0.10 to -0.05 at Qwen2.5-14B, while its post-marker loss remains (Appendix Table 14).

6.2 External Tasks

On tasks we did not design (Appendix Tables 18 and 19, Figures 10 and 11), the pattern holds. On CLRS-Text's running-minimum algorithm (Markeeva et al., 2026), where every position depends on the whole prefix, markers hurt wherever the model can do the task: -0.20 ($z = 4.5$) at Qwen2.5-32B and -0.34 ($z = 7.3$) on Gemini 3.5. On ProcBench's per-character substitution task (Fujisawa et al., 2024), a position-local task, markers help ($+0.10$, $z = 6.7$ at Qwen2.5-7B; $+0.20$, $z = 10.8$ at 32B); its strings mix letters and digits, so part of this gain may be tokenization. On string copying (Jelassi et al., 2024), where accuracy without markers is above 0.97, the effect is below the significance threshold ($z \leq 2.0$). Tagging each letter as a vowel or consonant from pretrained knowledge shows no benefit at Qwen2.5-32B ($+0.01$, $z = 0.5$); at 7B the baseline is at chance, so that cell is not interpretable.

7 Mechanistic Analysis

This section has three kinds of content, and we keep them apart: what the attention patterns *show*, what the ablations *establish* by intervention, and what we *conjecture* about the mechanism. The interventions are narrower than the rest of the paper: they are run on Qwen2.5 only, mainly on `random_permutation`, and they manipulate attention to markers rather than any other pathway. Read them as bounding what marker-attending heads contribute, not as a full account.

Working hypothesis. To produce output position i, a model must find input position i (and, on dependency tasks, position $i - 1$) among many near-identical tokens. We hypothesize that markers turn this global alignment into a local one: with a marker every k characters, the model can keep its place by the markers it has passed and its offset since the last one. This would explain a gain that appears only on long sequences, is spread over the whole segment, depends little on k, and shrinks when alignment slips are forgiven in scoring. The same local frame would explain the blind spot: if "the previous input" is looked up relative to the current segment, the first position of a segment finds a marker where its predecessor should be. This is a conjecture, not a result: the experiments below test only its simplest version, in which a few marker-attending heads carry the effects, and they rule that version out.

7.1 Attention-Head Ablations

Marker-attending heads. In Qwen2.5, individual heads attend sharply from each position to the nearest marker (Appendix Figures 12 and 13). Heads are ranked by the mean attention mass that all query positions place on the marker tokens, computed per (layer, head) over five held-out trials at the same task, length, and spacing and re-identified for every such cell; the top n are taken across all layers. For the ten heads with the most attention to markers at Qwen2.5-32B, the mean percentile on a previous-token test is 95 and on an induction-head test 31 (Appendix Table 22); since heads selected for attending to an adjacent token will score high on a previous-token test, we read this only as "these are local heads". The total attention mass on markers rises with density, but so does the number of markers, so we do not claim markers are attention sinks in the sense of drawing more attention than content tokens.

Blocking the ten most marker-attending heads. We block the ten most marker-attending heads from the markers during generation and compare with two controls (Table 4). The block acts after the softmax: in each blocked head, the attention weights from every query position to the marker tokens are set to zero and each row is renormalized, so the head's remaining attention is redistributed over non-marker tokens and nothing else in the forward pass changes; a self-test at the start of every run confirms that the blocked weights are exactly zero. On digits at Qwen2.5-32B, where markers raise accuracy from 0.41 to 0.96 at $N = 64$, blocking removes 1% of the gain (paired drop $+0.005 \pm 0.009$). At $N = 64$ it removes 13% on spaced letters